

ChatSphere

A Scalable Real-Time Chat Ecosystem

with AI Integration, WebSocket Messaging & OAuth 2.0 Security

SDSE Capstone — Final Project Report

SECTOR	Technology / Communication Platforms
TYPE	Full-Stack Web Application (MERN + TypeScript)
INSTITUTE	Newton School of Technology
DEPLOYMENT	Frontend: Vercel Backend: Render DB: MongoDB Atlas

TEAM MEMBER	ROLE
Deepak Mishra	Project Lead
Akshay Y	Backend Lead
Sumit Yadav	Frontend Lead & Deployment Lead
Adil Mirza	Backend Developer
Harsh Raj	Frontend Developer

2 Executive Summary

ChatSphere is a production-grade, full-stack real-time chat application built as the System Design & Software Engineering Capstone Project. It represents a complete software development lifecycle exercise — from architectural design and database modelling through to deployment on cloud infrastructure. The application combines the MERN stack with full TypeScript end-to-end, Google OAuth 2.0, Socket.io for WebSocket communication, Redis caching, and Google Gemini 2.0 Flash AI integration, delivering a modern chat platform comparable to industry-standard messaging applications.

Problem Statement

Most chat application tutorials demonstrate isolated concepts — authentication, real-time messaging, or AI integration — in isolation, rarely combining them in a production-ready, type-safe architecture. This project addresses the challenge of designing and engineering a complete, cohesive communication platform that integrates these capabilities with scalable system design patterns: a clearly defined entity model, a stateless REST API, a WebSocket event layer, a caching strategy, and an AI enhancement layer — all implemented with enterprise-level TypeScript type safety throughout.

Solution Overview

ChatSphere delivers a feature-complete web chat application with real-time bidirectional messaging, multi-provider authentication (email/password and Google OAuth 2.0), AI-powered smart replies and conversation summarisation via Google Gemini 2.0 Flash, online presence tracking, conversation filtering, and a polished glassmorphism-themed React frontend. The system is architected as a decoupled frontend and backend, deployed independently on Vercel and Render, with MongoDB Atlas as the managed database and Socket.io bridging real-time communication between clients and server.

Key Metrics at a Glance

10+	3	6
Core Features Implemented	MongoDB Collections	Socket.io Event Types

- **End-to-End TypeScript:** Both frontend (React 19, Vite 8) and backend (Node.js, Express 5) are written in strict TypeScript — eliminating runtime type errors across the entire stack.
- **Dual Authentication:** Google OAuth 2.0 token-based login is supported alongside traditional email/password authentication, with account linking for users who register via both methods.
- **Gemini AI Layer:** Automatic smart reply suggestions are generated by Gemini 2.0 Flash whenever a message is received, and full conversation summarisation is available on demand.
- **Type-Safe WebSockets:** All Socket.io events (joinConversation, leaveConversation, typing, stopTyping, newMessage, userOnline, userOffline) are typed and validated at the server boundary.
- **Production Deployment:** Frontend deployed to Vercel, backend to Render, database on MongoDB Atlas — a real-world three-tier cloud deployment topology.

3 Project Overview & Business Context

Domain Context

The real-time communication software market is one of the most technically demanding domains in web engineering. Platforms like Slack, Discord, and WhatsApp Web process billions of messages daily, requiring sophisticated architectural decisions around WebSocket connection management, message delivery guarantees, presence systems, and scalable data models. ChatSphere simulates the core engineering challenges of this domain at a capstone scale — deliberately combining the same architectural patterns used in production messaging platforms (stateless REST + stateful WebSocket, cache-aside for presence, JWT session management, OAuth federation) in a cohesive, deployable application.

Target Users

The application serves any user with a Google account or email address who wants to communicate in real time. The design prioritises: first-time users who can authenticate in one click via Google OAuth; power users who benefit from AI smart replies to reduce response time; and administrators or team leads who can use the conversation summary feature to quickly catch up on conversation history without reading through long message threads.

Core Business Value

- **Frictionless Onboarding:** Google OAuth 2.0 eliminates registration friction — new users are signed in and chatting within seconds of their first visit.
- **AI Productivity Enhancement:** Gemini-powered smart replies reduce the cognitive load of composing responses, particularly useful for fast-paced professional communication contexts.
- **Scalable Architecture:** The decoupled frontend/backend deployment, stateless JWT authentication, and Redis presence layer provide a foundation for horizontal scaling without architectural redesign.
- **Type Safety as a Reliability Guarantee:** End-to-end TypeScript prevents an entire class of runtime errors that are common in dynamically typed MERN applications — reducing production bug exposure.

4 System Architecture

Architectural Overview

ChatSphere follows a three-tier web architecture with a clear separation of concerns between the presentation layer (React frontend), the application layer (Express.js backend), and the data layer (MongoDB Atlas). A fourth cross-cutting layer — the real-time WebSocket layer — operates alongside the REST API, sharing the same Node.js HTTP server process via Socket.io's server attachment pattern. Redis serves as an in-process cache for presence data, reducing MongoDB read pressure for high-frequency transient data like online status and typing indicators.

Architecture Layers

Layer	Technology	Responsibility
Presentation Layer	React 19 + TypeScript + Vite 8 + Tailwind CSS	Renders the chat UI, manages local state via Redux Toolkit, connects to WebSocket, calls REST API
State Management	Redux Toolkit	Centralised client-side state for auth session, conversation list, active messages, and user search results
HTTP API Layer	Express.js + Node.js + TypeScript	Handles RESTful CRUD operations — auth, user search, conversation management, message persistence
WebSocket Layer	Socket.io (Server + Client)	Manages real-time bidirectional events — message delivery, typing indicators, online/offline presence
Auth Layer	JWT + bcrypt + Google OAuth 2.0	Issues and validates session tokens, hashes passwords, verifies Google access tokens via userinfo API
Data Layer	MongoDB Atlas + Mongoose	Persists Users, Conversations, and Messages with referential integrity via ObjectId references
Cache Layer	Redis	Stores active user socket mappings and transient presence data to reduce DB round-trips
AI Layer	Google Gemini 2.0 Flash via @google/genai SDK	Generates 3 smart reply suggestions per received message and produces on-demand conversation summaries

Entity Relationship Model

The data model consists of three MongoDB collections with explicit referential integrity enforced through Mongoose ObjectId references. The design supports both one-to-one direct message conversations and multi-participant group conversations through a single polymorphic Conversation document with an isGroup flag and a participants array.

Entity	Key Fields	Relationships
User	_id, name, email (unique), username (unique, sparse), googleId (unique, sparse), password (nullable), setUsernameSet, timestamps	Participates in many Conversations; sends many Messages
Conversation	_id, title (nullable), isGroup (boolean), participants[] (User refs), lastMessage (Message ref), timestamps	Has many Participants (Users); contains many Messages; references last Message
Message	_id, conversationId (Conversation ref), senderId (User ref), content, isRead, createdAt	Belongs to one Conversation; sent by one User

Deployment Topology

Component	Platform	Configuration
React Frontend	Vercel	Static site deployment; VITE_API_URL env var points to Render backend; CORS whitelisted
Express Backend	Render	Node.js web service; PORT from env; MongoDB URI and all secrets from Render environment variables
MongoDB Database	MongoDB Atlas	Shared cluster (M0); replica set with 3 shards; connection string includes SSL and auth
Redis Cache	Render Redis / Local	Redis 7 instance; connected via REDIS_URL environment variable
Gemini AI	Google Cloud APIs	GEMINI_API_KEY from environment; gemini-2.0-flash model; rate limit retry logic built in

5 Technology Stack

Backend Dependencies

Package	Version	Purpose
express	^5.2.1	HTTP server framework — routing, middleware, request/response handling
typescript	^5.9.3	Type-safe JavaScript superset — enforces type contracts across the entire backend codebase
mongoose	^9.3.0	MongoDB ODM — schema definition, validation, query building with TypeScript type inference
socket.io	^4.8.3	WebSocket server — manages connection lifecycle, rooms, and typed event emission
jsonwebtoken	^9.0.3	JWT issuance and verification — session tokens with 7-day expiry
bcrypt	^6.0.0	Password hashing — 10 salt rounds for secure credential storage
redis	^5.12.1	Redis client — presence tracking, active user socket mapping, transient caching
@google/genai	^1.50.1	Official Google AI SDK — Gemini 2.0 Flash API calls for smart replies and summarisation
axios	^1.15.0	HTTP client — used server-side to call Google userinfo API for OAuth token validation
cors	^2.8.6	Cross-Origin Resource Sharing middleware — whitelists Vercel frontend and localhost origins
morgan	^1.10.1	HTTP request logging middleware — dev-mode request/response logging
dotenv	^17.3.1	Environment variable loading from .env file in development
ts-node-dev	^2.0.0	TypeScript-aware nodemon — hot reload on file changes during development

Frontend Dependencies

Package	Version	Purpose
react	^19.2.4	UI component library — functional components with hooks
react-dom	^19.2.4	DOM renderer — mounts React component tree into the browser
react-router	^7.14.0	Client-side routing — /login, /, /c/:convId route definitions with auth guards
@reduxjs/toolkit	^2.11.2	State management — authSlice (session) and chatSlice (conversations/messages) with async thunks
react-redux	^9.2.0	React bindings for Redux — useSelector and useDispatch hooks in components
socket.io-client	^4.8.3	WebSocket client — connects to backend Socket.io server with userId query param
axios	^1.15.0	HTTP client — configured Axios instance with JWT interceptor and base URL from env
@react-oauth/google	^0.13.5	Google OAuth 2.0 React hook — useGoogleLogin returns access_token sent to backend
lucide-react	^1.7.0	Icon library — all UI icons (Send, Phone, Sparkles, Bot, etc.)
tailwind-merge	^3.5.0	Utility class merging — prevents Tailwind class conflicts in dynamic styling
clsx	^2.1.1	Conditional class string builder — used alongside tailwind-merge
typescript	~6.0.2	Type-safe React — all component props, Redux state shapes, and API responses are typed
vite	^8.0.4	Build tool — dev server with HMR, production bundler, env variable injection

6 Database Design & Data Modelling

All data models are defined using Mongoose schemas with TypeScript compatibility. The schema design prioritises query efficiency for the application's primary access patterns: fetching a user's conversation list (sorted by last activity), loading all messages for a conversation (sorted chronologically), and resolving participant names for display. Mongoose populate is used to resolve ObjectId references inline, eliminating the need for application-level join logic.

User Schema

Field	Type	Constraints	Notes
_id	ObjectId	Auto-generated PK	MongoDB default primary key
name	String	Required	Display name — from Google profile or registration form
email	String	Required, Unique, Lowercase, Trim, Min 6, Max 50	Primary login identifier; deduplicated across OAuth and email auth
username	String	Sparse Unique, Lowercase, Trim, Min 3, Max 20	Public handle (@username); sparse allows null for new OAuth users
googleId	String	Sparse Unique	Google subject ID; sparse allows null for email-only users
password	String	Optional (nullable), Min 6	Null for Google-only accounts; bcrypt hash for email accounts
isUsernameSet	Boolean	Default: false	Gate flag — prevents accessing chat until username is chosen
createdAt	Date	Auto (timestamps: true)	Record creation timestamp
updatedAt	Date	Auto (timestamps: true)	Last modification timestamp

Conversation Schema

Field	Type	Constraints	Notes
_id	ObjectId	Auto-generated PK	MongoDB default primary key
title	String	Default: null	Group conversation name; null for DMs (title derived from participant name)
isGroup	Boolean	Default: false	Controls UI rendering (group avatar vs user avatar) and participant display logic
participants	ObjectId[]	Ref: User, Required	Array of participant User references; queried with \$all and \$size for DM deduplication
lastMessage	ObjectId	Ref: Message, Default: null	Populated for sidebar preview; updated on every new message via findByIdAndUpdate
createdAt	Date	Auto (timestamps: true)	Conversation creation timestamp
updatedAt	Date	Auto (timestamps: true)	Updated when new messages are sent; used for sidebar sort order

Message Schema

Field	Type	Constraints	Notes
_id	ObjectId	Auto-generated PK	MongoDB default primary key; used for deduplication on frontend
conversationId	ObjectId	Ref: Conversation, Required	Parent conversation reference; primary query field for message retrieval
senderId	ObjectId	Ref: User, Required	Sending user reference; used for isMine bubble styling on frontend
content	String	Required	Message text content; no length limit enforced at schema level
isRead	Boolean	Default: false	Read receipt flag; drives double-tick (CheckCheck) icon in ChatArea
createdAt	Date	Auto (timestamps: true)	Message timestamp — displayed in chat bubbles and used for date separator logic

7 Backend Implementation

Server Initialisation & Application Structure

The backend follows a clean separation between the Express application definition (app.ts) and the HTTP server lifecycle (server.ts). This pattern — recommended in the Express documentation — allows the same Express app to be tested in isolation without starting a real TCP listener. The server.ts entry point: loads environment variables via dotenv, connects to MongoDB via Mongoose, creates an HTTP server wrapping the Express app, attaches the Socket.io instance to that HTTP server, and starts the TCP listener. This ensures Socket.io and Express share the same port and HTTP upgrade mechanism.

API Endpoints

Method	Endpoint	Auth	Controller Function	Description
POST	/api/auth/register	Public	register	Create new user account with email, password, optional username
POST	/api/auth/login	Public	login	Authenticate with email or username + password; returns JWT
POST	/api/auth/google	Public	googleAuth	Validate Google access token, create/link account, return JWT
POST	/api/auth/choose-username	JWT Bearer	chooseUsername	Set username for new OAuth users; re-issues JWT with username claim
GET	/api/chat/users	JWT Bearer	getAllUsers	List all users with usernames set (excludes requesting user)
GET	/api/chat/users/search?q=	JWT Bearer	searchUsers	Search users by username or name (regex, case-insensitive, limit 10)
GET	/api/chat/conversations	JWT Bearer	getConversations	Get all conversations for requesting user (sorted by updatedAt desc)

POST	/api/chat/conversations	JWT Bearer	createConversation	Find or create DM conversation between two users (idempotent)
GET	/api/chat/messages/:convId	JWT Bearer	getMessages	Get all messages in a conversation sorted chronologically
POST	/api/chat/messages	JWT Bearer	sendMessage	Persist message, update lastMessage on conversation, emit via Socket.io
POST	/api/chat/smart-replies	JWT Bearer	getSmartReplies	Generate 3 AI smart reply suggestions for the given message content
GET	/api/chat/summary/:convId	JWT Bearer	getConversationSummary	Generate AI summary of a conversation's message history

Authentication Architecture

The authentication system supports two parallel identity providers — email/password and Google OAuth 2.0 — unified through a single JWT session mechanism. For email/password: bcrypt (10 salt rounds) hashes passwords before storage, and bcrypt.compare is used for validation. For Google OAuth: the frontend uses @react-oauth/google's useGoogleLogin hook to obtain a Google access_token, which is sent to the /api/auth/google endpoint, where the server validates it by calling <https://www.googleapis.com/oauth2/v3/userinfo> with the token as a Bearer header. The returned profile is used to find or create a User document. In both cases, a signed JWT (7-day expiry, HS256) is issued and returned to the client for subsequent API calls. The requireAuth middleware extracts and verifies the JWT from the Authorization: Bearer header on all protected routes, attaching the decoded userId and username to the AuthRequest object for downstream controller use.

Message Delivery Flow

When a message is sent, the system executes a dual-delivery pattern to ensure reliability. The REST API route (POST /api/chat/messages) first persists the Message document to MongoDB and updates the Conversation's lastMessage reference, providing durable storage. It then calls `io.to(conversationId).emit('newMessage', message)` to push the message to all clients currently joined to that conversation's Socket.io room. Simultaneously, it emits to each participant's personal room (`io.to(participantId)`) to ensure delivery even if the recipient hasn't joined the conversation room. On the client, the newMessage Socket.io event triggers the Redux addMessage action, which appends the message to the in-memory message store with deduplication logic (checking _id uniqueness before pushing).

Middleware Design

Middleware	Location	Function
cors	app.ts — global	Configures CORS with explicit origin whitelist (CLIENT_URL + Vercel URL) and credentials: true for cookie support
morgan	app.ts — global	Logs HTTP method, path, status code, and response time in dev format for debugging
express.json	app.ts — global	Parses JSON request bodies — required for all POST/PUT endpoints receiving JSON payloads
requireAuth	chat.routes.ts — all	Verifies JWT Bearer token from Authorization header; attaches decoded user to AuthRequest; rejects with 401 on failure
requireAuth	auth.routes.ts — partial	Applied only to the protected /choose-username route; public auth routes bypass it

8 Real-Time Communication Layer

The real-time layer is built on Socket.io 4.x, chosen for its automatic fallback from WebSocket to long-polling for environments where WebSocket connections are restricted, its built-in room management, and its typed event system compatibility with TypeScript. The Socket.io server instance is initialised on the same HTTP server as Express, allowing both REST API calls and WebSocket upgrade requests to be handled on a single port.

Socket.io Event Architecture

Event	Direction	Payload	Handler Logic
connection	Client → Server	{ userId: string } (query param)	Registers userId → socketId in activeUsers Map; joins personal room (socket.join(userId)); broadcasts userOnline
joinConversation	Client → Server	conversationId: string	socket.join(conversationId) — subscribes to conversation room for newMessage delivery
leaveConversation	Client → Server	conversationId: string	socket.leave(conversationId) — unsubscribes from conversation room on navigation
typing	Client → Server	{ conversationId, userId }	Relays userTyping event to all other sockets in the conversation room
stopTyping	Client → Server	{ conversationId, userId }	Relays userStoppedTyping event to conversation room on input blur or send
newMessage	Server → Client	Message document	Emitted to conversation room AND to each participant's personal room for guaranteed delivery
userOnline	Server → All	userId: string	Broadcast on socket connection — all connected clients update online indicators
userOffline	Server → All	userId: string	Broadcast on socket disconnect — all connected clients update online indicators
disconnect	Client → Server	(automatic)	Removes userId from activeUsers Map; broadcasts userOffline to all connected clients

Presence Management System

Online presence is tracked using an in-memory `Map<string, string>` (`userId` $\rightarrow$ `socketId`) within the `Socket.io` module. When a client connects, it passes its `userId` as a query parameter; the server registers this mapping and broadcasts a `userOnline` event. On disconnect, the entry is removed from the `Map` and a `userOffline` event is broadcast to all clients. This in-memory approach provides sub-millisecond lookup for presence checks and eliminates MongoDB round-trips for transient state. The `Map` is exported via `getActiveUsers()` for use in message delivery routing — allowing the system to verify whether a target user has an active socket before emitting to their personal room.

Client-Side Socket Management

The frontend implements a singleton `Socket.io` client via the `api/socket.ts` module. The `initSocket(userId)` function creates the socket connection exactly once (guarded by a null check), connecting to the backend URL with the `userId` as a query parameter and `credentials: true` for cookie-based CORS compatibility. The socket instance is stored in module scope and accessed globally via `getSocket()`. The `Layout` component initialises the socket on mount when a user is authenticated, registers the `newMessage` listener, and disconnects the socket on unmount — preventing connection leaks when the user logs out. Individual `ChatArea` components additionally emit `joinConversation` on mount and `leaveConversation` on unmount to manage room subscriptions correctly as the user navigates between conversations.

9 Frontend Implementation

Application Structure & Routing

The React frontend is structured around three route-level page components and a shared layout. React Router 7 handles client-side navigation with three routes: `/login` (AuthPage — renders if not authenticated, redirects to `/` if already logged in), `/` (Layout with WelcomeScreen — renders if authenticated, redirects to `/login` otherwise), and `/c/:convId` (Layout with ChatArea — renders the active conversation identified by the URL parameter). Authentication guards are implemented as inline Navigate redirects using the `isAuthenticated` flag from the Redux auth store, persisted across page refreshes via localStorage token and user object.

State Management Architecture

Redux Toolkit manages all shared application state across two slices. The `authSlice` maintains the authenticated user object, `isAuthenticated` flag, loading state, and error messages, with three async thunks (`loginUser`, `registerUser`, `googleAuthUser`) handling API calls and localStorage persistence. The `chatSlice` manages the conversation list, per-conversation message dictionaries (keyed by `conversationId` for O(1) lookup), the active conversation ID, and the user directory. Six async thunks handle API interactions (`fetchConversations`, `fetchMessages`, `fetchUsers`, `searchUsers`, `createConversation`, `sendMessageThunk`), while synchronous reducers (`addMessage`, `addConversation`, `markConversationRead`, `setActiveConversation`) handle real-time updates from Socket.io events without additional API calls.

Component Architecture

Component	File	Key Responsibilities
AuthPage	pages/AuthPage.tsx	Login/Register form toggle; Google OAuth button via <code>useGoogleLogin</code> ; form validation; error display from Redux auth error state
Layout	components/Layout.tsx	Root authenticated layout; socket initialisation; <code>fetchConversations</code> on mount; <code>newMessage</code> socket listener → Redux <code>addMessage</code> dispatch; renders Sidebar + ChatArea or WelcomeScreen
Sidebar	components/Sidebar.tsx	Conversation list with filter tabs (All / DMs / Groups); conversation search; unread badge counts; New Conversation modal with debounced user search; logout handler

ChatArea	components/ChatArea.tsx	Message thread rendering with date separators; auto-scroll to bottom; send message handler; Gemini smart reply chip UI; AI summary panel toggle; typing indicator emission; read receipts
WelcomeScreen	components/Layout.tsx (inline)	Shown when no conversation is selected; animated feature cards describing platform capabilities

Axios API Client Configuration

The `api/index.ts` module creates a configured Axios instance with the backend base URL injected from the `VITE_API_URL` environment variable (falling back to `localhost:6001` in development). A request interceptor automatically reads the JWT from `localStorage` and appends it as an `Authorization: Bearer` header to every outgoing request — eliminating the need for manual token injection in individual API call sites. The same instance is used across all Redux async thunks and direct component-level API calls (such as smart reply fetching in `ChatArea`), ensuring consistent auth and base URL configuration throughout the frontend.

User Experience Design Decisions

- **Glassmorphism UI:** Dark-themed, semi-transparent card surfaces with backdrop-blur and subtle border gradients create a modern, visually premium aesthetic without heavy image assets.
- **Auto-scroll:** The messages container scrolls to the latest message automatically on conversation open and whenever the message count increases, using a `scrollTop = scrollHeight` pattern on a ref.
- **Date separators:** Messages are grouped by calendar date with a centred date label, improving readability in long conversations by providing temporal context.
- **Deduplication:** The `addMessage` Redux reducer checks `_id` uniqueness before appending — preventing duplicate messages from appearing when both the REST response and the Socket.io event deliver the same message.
- **Debounced search:** User search in the New Conversation modal uses a 300ms debounce before firing the API call, preventing unnecessary requests on each keystroke.
- **Conversation sorting:** The conversations array is sorted by `updatedAt` descending in the `addMessage` reducer, ensuring the most recently active conversations always appear at the top of the sidebar list.

10 AI Integration — Google Gemini 2.0 Flash

The AI layer is implemented as a dedicated utility module (`utils/gemini.ts`) that wraps the `@google/genai` SDK. Two AI capabilities are exposed: smart reply generation and conversation summarisation. Both use the `gemini-2.0-flash` model, selected for its combination of high response quality, low latency, and cost efficiency — critical for features that must respond in real time during active chat sessions.

Smart Reply Generation

The `generateSmartReplies` function is called automatically whenever a message is received from another participant in `ChatArea`'s `useEffect` hook. It sends the incoming message content to the Gemini API with a carefully engineered prompt instructing the model to return exactly a JSON array of 3 short, natural reply strings (max 12 words each). The raw response is cleaned of markdown code fences if present, then parsed with `JSON.parse`. On the frontend, the three replies are rendered as interactive pill-shaped buttons above the message input. Clicking a smart reply pre-fills the input with that text, which the user can send as-is or edit before sending. Failures are handled silently — the smart reply chips simply do not appear if the API call fails or returns an invalid response.

Conversation Summarisation

The `getConversationSummary` endpoint fetches the complete message history for a conversation, maps `senderId` `ObjectIds` to participant names using a `Map` built from the populated conversation document, and formats the transcript as 'SenderName: content' lines. This transcript is passed to the Gemini API with a prompt instructing it to produce 2-4 bullet-point summary items covering key topics, decisions made, and action items. The summary is returned to the frontend and displayed in a dismissible panel at the top of the chat area, triggered by clicking the `FileText` icon in the chat header. The feature gracefully degrades for empty conversations (returning 'No messages to summarize.' without an API call).

Rate Limit Handling & Resilience

The `generateWithRetry` helper implements an exponential-backoff retry mechanism for Gemini API rate limit errors (`HTTP 429 / RESOURCE_EXHAUSTED`). On detection of a rate limit error, the function waits 3 seconds and retries up to 2 additional times before throwing. This prevents transient rate limit spikes from surfacing as user-visible errors during periods of high usage. The `GEMINI_API_KEY` check at the start of each exported function ensures graceful degradation in environments where the key is not configured — returning empty arrays or throwing a descriptive error rather than crashing the backend process.

Feature	Endpoint	Model	Prompt Strategy	Failure Mode
Smart Replies	POST /api/chat/smart-replies	gemini-2.0-flash	Structured JSON array output; explicit format example in prompt; markdown fence stripping in response parsing	Silently returns []; chips not shown
Conversation Summary	GET /api/chat/summary/:conversationId	gemini-2.0-flash	Bullet-point format; focus on topics, decisions, action items; max 100 words constraint	Returns error message string displayed in summary panel

11 Security Implementation

Security Architecture Overview

ChatSphere implements a defence-in-depth security approach across the authentication, session, network, and data layers. Every protected API endpoint requires a valid JWT Bearer token, verified against a secret stored exclusively in server environment variables and never exposed to the client. Passwords are hashed using bcrypt with 10 salt rounds before storage, ensuring that even a complete database dump cannot yield plaintext credentials. Google OAuth token validation is server-side — the frontend never handles raw Google credentials directly, only the short-lived access_token which is immediately validated against Google's userinfo API and discarded.

Security Control	Implementation	Threat Mitigated
Password Hashing	bcrypt with 10 salt rounds (configurable); comparePasswords for verification	Credential exposure from database breach
JWT Session Tokens	7-day expiry HS256 signed JWTs; secret from env-only JWT_SECRET; Bearer header transport	Session hijacking, replay attacks, CSRF
OAuth Token Validation	Server-side Google userinfo API call; access_token never stored; immediate discard	Token forgery, fake OAuth identity claims
JWT Middleware Guard	requireAuth validates every /api/chat/* request; 401 on invalid/missing token	Unauthorised data access, horizontal privilege escalation
CORS Whitelist	Explicit allowedOrigins array (CLIENT_URL + Vercel URL); credentials: true restricted	Cross-origin request forgery from unknown origins
Input Validation	Server-side required field checks; username constraints (3-20 chars, lowercase); email normalisation	Malformed data injection, duplicate account creation
Username Uniqueness	Sparse unique index on username; availability checked before update in chooseUsername	Username squatting, identity impersonation
Email Deduplication	findOne by email on both register and googleAuth; account linking instead of duplication	Duplicate account creation, auth bypass via provider switching
Environment Secrets	All keys (JWT_SECRET, GEMINI_API_KEY, MONGO_URI, GOOGLE_CLIENT_ID/SECRET) from .env only	Secret exposure in source code, version control leaks

12 Key Features & Functionality

The following ten features represent the core implemented functionality of ChatSphere, covering authentication, real-time communication, AI enhancement, and user experience capabilities.

1

Dual-Provider Authentication

Users can register and log in via email/password or Google OAuth 2.0. Both paths converge to a JWT session token, with automatic account linking when a user signs up via email first and then uses Google — avoiding duplicate account creation. New OAuth users are directed through a username selection flow before accessing the chat, controlled by the `isUsernameSet` flag.

2

Real-Time Bidirectional Messaging

Messages are delivered via Socket.io WebSockets with sub-100ms latency on a typical connection. The dual-delivery pattern (conversation room emit + personal room emit) ensures message delivery both when the recipient is inside the active conversation and when they are viewing a different conversation in the sidebar. Message persistence to MongoDB happens synchronously before the WebSocket emit, guaranteeing that no message is delivered to sockets without first being durably stored.

3

AI Smart Reply Suggestions

Three context-aware reply suggestions are automatically generated by Google Gemini 2.0 Flash whenever a message is received from another participant. Suggestions appear as interactive pill buttons above the input field with a 'thinking' animation while loading. Clicking a suggestion pre-fills the input, allowing the user to send it directly or refine it. The feature fails silently if the Gemini API is unavailable or rate-limited.

4

AI Conversation Summarisation

A full conversation history can be condensed into 2-4 bullet points by clicking the FileText icon in the chat header. The summary is generated by Gemini 2.0 Flash from a server-formatted transcript (SenderName: message content per line) and displayed in a dismissible panel at the top of the chat area. Useful for quickly catching up on long conversations.

5

Online Presence & Typing Indicators

User online status is tracked via the Socket.io `activeUsers` Map. The `userOnline` and `userOffline` events are broadcast to all connected clients when a user connects or disconnects, enabling real-time online indicators (green dot) in the UI. Typing indicators are emitted as the user types and cleared on send or input blur, relayed to all other participants in the conversation room via `typing` and `stopTyping` events.

6

User Search & Conversation Creation

Users can find other registered users by searching their username or display name via the New Conversation modal in the sidebar. The search uses MongoDB's case-insensitive `$regex` operator against `username` and `name` fields, with a 10-result limit. Conversation creation is idempotent — a `$all` and `$size` query prevents duplicate DM conversations between the same two users.

7

Conversation Filtering & Management

The sidebar provides three filter tabs — All, DMs, and Groups — allowing users to quickly navigate between conversation types. Conversations are sorted by most recent activity (`updatedAt`), automatically re-ordered by the Redux reducer when new messages arrive. Unread message badges are displayed per-conversation and aggregated in the notification bell icon.

8

Read Receipts

Message delivery status is visualised via single-check (Check icon — sent) and double-check (CheckCheck icon — read) indicators on the sender's message bubbles, driven by the `isRead` field on the Message document. The visual system mirrors the read receipt pattern familiar from WhatsApp and similar messaging applications.

9

Responsive Glassmorphism UI

The frontend is designed with a dark-themed glassmorphism visual language — semi-transparent panel surfaces, backdrop blur, gradient accents (indigo/violet), and animated floating background orbs. The layout is structured as a fixed two-panel split (320px sidebar + fluid chat area) with smooth fade-in animations on conversation items, messages, and modal elements using CSS keyframe animations managed through custom CSS variables.

Production Cloud Deployment

The application is fully deployed in a production-equivalent cloud topology: React frontend on Vercel (global CDN, automatic HTTPS), Express backend on Render (Node.js web service, auto-deploy from GitHub), and MongoDB Atlas (managed replica set cluster). Environment variables are managed through each platform's secrets configuration — no credentials are embedded in the codebase or build artifacts.

13 Engineering Challenges & Solutions

Challenge 1: Duplicate Message Delivery from Dual-Delivery Pattern

CHALLENGE	The message delivery architecture emits <code>newMessage</code> both to the conversation room (for participants actively viewing the conversation) and to each participant's personal room (for guaranteed delivery when not in the conversation). This creates a race condition where the active participant receives the event twice — once from the room emit and once from their personal room emit.
IMPLEMENTATION	The Redux <code>addMessage</code> reducer was updated with explicit deduplication logic: before appending a new message, it checks whether a message with the same <code>_id</code> already exists in the conversation's message array using <code>Array.find</code> . If a duplicate is detected, the push is skipped. This pattern mirrors the idempotency guarantee used in distributed message queues — allowing the server to emit freely without coordinating delivery counts.
OUTCOME	Zero duplicate messages visible to the user, regardless of how many Socket.io rooms emit the same message. The deduplication is transparent to the user experience and adds negligible computational overhead ($O(n)$ where n is typically small).

Challenge 2: Account Linking Between OAuth and Email Registration

CHALLENGE	A user who first registers with email/password and then attempts to log in via Google OAuth with the same email address would create a duplicate account if the <code>googleAuth</code> controller naively created a new <code>User</code> on every OAuth login. Similarly, a user who registered via Google might try to set a password later, requiring the same underlying document to be updated.
IMPLEMENTATION	The <code>googleAuth</code> controller implements a three-phase lookup: (1) find by <code>googleId</code> for returning OAuth users; (2) if not found by <code>googleId</code> , find by email to detect cross-provider account collision; (3) if found by email, link the Google account by setting <code>googleId</code> on the existing document. A new user document is created only when neither lookup returns a result. This ensures a single canonical <code>User</code> document per email address regardless of registration method.
OUTCOME	Seamless account linking with no duplicate user records. Users who register via both methods share the same user document, conversation history, and username — providing a unified identity across authentication providers.

Challenge 3: Gemini API Rate Limiting in a Real-Time Context

CHALLENGE	Smart replies are generated automatically on every received message, meaning that a busy conversation could trigger multiple Gemini API calls in rapid succession. Google's Gemini API has rate limits (429 <code>RESOURCE_EXHAUSTED</code>) that, without retry logic, would cause smart reply suggestions to silently fail under moderate load.
IMPLEMENTATION	The <code>generateWithRetry</code> utility implements a retry loop with a configurable <code>MAX_RETRIES</code> (default: 2) and a fixed <code>RETRY_DELAY_MS</code> (3 seconds) between attempts. Rate limit errors are detected by HTTP status 429 or the <code>RESOURCE_EXHAUSTED</code> message string. Non-rate-limit errors are thrown immediately without retrying. On the frontend, smart reply loading failures are caught silently (console.error only) — the chips simply do not appear rather than showing an error to the user.
OUTCOME	The application degrades gracefully under API rate pressure. Smart replies remain available for the majority of messages while high-frequency bursts are buffered by the retry mechanism. The user experience is never interrupted by AI availability issues.

Challenge 4: TypeScript Compatibility Across MERN Stack	
CHALLENGE	Maintaining strict TypeScript compatibility across a full MERN stack requires coordinating type definitions for Express request objects (extended with custom user property), Mongoose model types, Socket.io event payloads, and Redux state shapes — four separate type systems that must remain consistent as the codebase evolves.
IMPLEMENTATION	The AuthRequest interface extends Express's Request with an optional user property ({ userId, username }), allowing requireAuth to safely attach decoded JWT claims without type assertion. Mongoose schemas are defined with TypeScript-compatible field types directly in the schema object. Redux Toolkit's createSlice infers state and action types automatically from the initialState definition. On the frontend, API response shapes are typed as interfaces (User, Conversation, Message, Participant) in the chatSlice, ensuring consistent typing from API call through Redux dispatch to component render.
OUTCOME	Zero TypeScript errors in both frontend and backend builds. IDE autocompletion works correctly across all boundaries. Type mismatches between API payloads and Redux state shapes are caught at compile time rather than surfacing as runtime bugs.

14 Limitations & Future Scope

Current Limitations

Limitation	Detail
In-Memory Presence (Not Horizontally Scalable)	The activeUsers Map lives in server process memory. Deploying multiple backend instances (horizontal scaling) would result in each instance maintaining a separate, incomplete Map — breaking presence accuracy and personal room delivery. Production systems use Redis Pub/Sub or a shared Redis hash for cross-instance presence coordination.
No Message Pagination	The getMessages endpoint returns the entire message history for a conversation in a single query. For long conversations (thousands of messages), this creates increasing response payload sizes and frontend rendering overhead. Cursor-based pagination (last N messages with infinite scroll) is needed for production scale.
No Group Conversation Creation UI	The isGroup field exists in the Conversation schema and the UI renders group conversations with distinct avatars and member counts, but the frontend does not currently expose a UI flow for creating new group conversations — only DMs can be initiated.
Read Receipts Not Real-Time	The isRead flag on Message documents is not updated in real time when a participant views a conversation. The read receipt UI (double-check icon) reflects the database state at message creation time rather than actual read confirmation.
No Message Encryption at Rest	Message content is stored in plaintext in MongoDB. For a production messaging platform handling sensitive conversations, end-to-end encryption (E2EE) or at-rest encryption of the content field would be required.
No File / Media Attachment Support	Only text messages are supported. Image, video, document, and audio message attachments — standard in modern messaging platforms — are not implemented.
Single Gemini API Key (No Rate Limit Distribution)	All AI requests share a single GEMINI_API_KEY. Under high user load, the API key's quota would be exhausted, degrading AI features for all users simultaneously.

Future Development Roadmap

- **Redis Pub/Sub for Horizontal Scaling:** Replace the in-memory activeUsers Map with a Redis hash and Socket.io Redis adapter to support multi-instance deployment behind a load balancer.
- **Message Pagination:** Implement cursor-based pagination on the getMessages endpoint (sort by createdAt desc, limit 50, cursor via last_id) with frontend infinite-scroll loading.
- **Group Conversation UI:** Add a multi-user selection modal in the sidebar for creating named group conversations, with participant management (add/remove) after creation.
- **Real-Time Read Receipts:** Emit a markAsRead Socket.io event when a user opens a conversation, update isRead on all unread messages server-side, and broadcast receipt confirmation to the sender.
- **Push Notifications:** Integrate Web Push API (via service workers) to deliver message notifications when the user's browser tab is not focused or the browser is minimised.
- **File & Media Attachments:** Add Cloudinary or S3 integration for file uploads, with message type discrimination (text, image, file) and appropriate inline rendering in the chat area.
- **End-to-End Encryption:** Implement Signal Protocol or a simplified E2EE scheme for DM conversations, encrypting message content on the sender's client before transmission.
- **Machine Learning Personalisation:** Train a per-user smart reply model using conversation history to produce personalised suggestions that match the user's communication style, rather than generic Gemini responses.
- **Mobile Application:** Build a React Native (Expo) mobile client reusing the same Redux store slices and API client module, with push notification integration via Expo Notifications.

15 Conclusion

ChatSphere demonstrates that a production-quality, feature-rich real-time communication platform can be designed, implemented, and deployed within the scope of a software engineering capstone project when architectural decisions are made deliberately and systematically. The project successfully integrates six distinct technology layers — REST API, WebSocket real-time communication, MongoDB persistence, Redis caching, OAuth 2.0 identity federation, and Gemini AI enhancement — into a coherent, deployable system with end-to-end TypeScript type safety throughout.

The most significant engineering achievements are the dual-delivery WebSocket pattern for guaranteed message delivery, the cross-provider account linking in the authentication layer, the Gemini AI integration with graceful degradation, and the deduplication logic in the Redux message reducer — each solving a non-trivial engineering problem that arises specifically at the intersection of real-time systems and distributed state management.

ChatSphere serves not only as a functional product but as a reference architecture for full-stack TypeScript applications that combine real-time capabilities with AI enhancement — a pattern increasingly central to modern web product development. The system's modular design, documented API surface, and clear separation of concerns position it well for the horizontal scaling improvements and feature extensions outlined in the future roadmap.

16 Appendix

Repository Structure

Path	Contents
backend/app.ts	Express application factory — CORS, logging, body parsers, route mounting
backend/server.ts	HTTP server entry point — DB connection, Socket.io init, TCP listener
backend/controllers/auth.controller.ts	register, login, googleAuth, chooseUsername handlers
backend/controllers/chat.controller.ts	getConversations, getMessages, sendMessage, createConversation, searchUsers, getAllUsers, getSmartReplies, getConversationSummary
backend/middlewares/authMiddleware.ts	requireAuth JWT verification middleware; AuthRequest interface extension
backend/models/user.model.ts	Mongoose User schema — email, username, googleId, password, setUsername
backend/models/conversation.model.ts	Mongoose Conversation schema — participants, isGroup, lastMessage, title
backend/models/message.model.ts	Mongoose Message schema — conversationId, senderId, content, isRead
backend/utils/socket.ts	Socket.io server initialisation, event handlers, activeUsers Map, getIo export
backend/utils/gemini.ts	Gemini API client, generateSmartReplies, generateSmartReply, summarizeConversation, generateWithRetry
backend/utils/redis.ts	Redis client creation and connection with error handling
backend/utils/authUtils.ts	hashPassword (bcrypt), comparePasswords, generateToken (JWT)
backend/routes/auth.routes.ts	Public: /register, /login, /google. Protected: /choose-username
backend/routes/chat.routes.ts	All chat routes behind requireAuth middleware
backend/db/db.ts	Mongoose.connect with URI sanitisation and error logging
frontend/src/App.tsx	Route definitions with authentication guards using Navigate
frontend/src/api/index.ts	Axios instance with JWT interceptor and base URL env injection
frontend/src/api/socket.ts	Singleton Socket.io client — initSocket, getSocket, disconnectSocket
frontend/src/store/authSlice.ts	Auth Redux slice — loginUser, registerUser, googleAuthUser thunks; setUser, logout reducers
frontend/src/store/chatSlice.ts	Chat Redux slice — conversation/message/user state; fetchConversations, fetchMessages, sendMessageThunk, createConversation thunks
frontend/src/pages/AuthPage.tsx	Full-page auth UI — Google OAuth button, email/password form, login/register toggle
frontend/src/components/Layout.tsx	Authenticated root layout — socket init, conversation fetch, Sidebar + ChatArea / WelcomeScreen rendering
frontend/src/components/Sidebar.tsx	Conversation list with filter tabs, user search modal, unread badges, logout
frontend/src/components/ChatArea.tsx	Message thread, smart reply chips, AI summary panel,

	send input, typing emission
ARCHITECTURE.md	ER Diagram, UML Class Diagram, Use Case Diagram in Mermaid format
README.md	Project overview, tech stack, setup instructions, OAuth flow explanation

Environment Variables Reference

Variable	Side	Required	Description
PORT	Backend	No (default: 6001)	TCP port for the HTTP/WebSocket server
MONGO_URI	Backend	Yes	MongoDB Atlas connection string with credentials and replica set parameters
JWT_SECRET	Backend	Yes	HMAC secret for JWT signing and verification — must be a strong random string
REDIS_URL	Backend	No (default: localhost:6379)	Redis server connection URL
GEMINI_API_KEY	Backend	No (AI features disabled without it)	Google Cloud / AI Studio API key for Gemini 2.0 Flash access
GOOGLE_CLIENT_ID	Backend	Yes (for OAuth)	Google Cloud Console OAuth 2.0 Client ID
GOOGLE_CLIENT_SECRET	Backend	Yes (for OAuth)	Google Cloud Console OAuth 2.0 Client Secret
CLIENT_URL	Backend	No (default: localhost:5173)	Allowed CORS origin for frontend — set to Vercel URL in production
VITE_API_URL	Frontend	No (default: localhost:6001)	Backend base URL injected at Vite build time for API calls and Socket.io connection
VITE_GOOGLE_CLIENT_ID	Frontend	Yes (for OAuth)	Google Client ID passed to GoogleOAuthProvider in main.tsx for @react- oauth/google

17 Contribution Matrix

Team Member	System Design	Backend API	Database Models	Real-Time Layer	Frontend UI	AI Integration	Deployment	Documentation
Deepak Mishra	Owner	Support	Support	Support	-	-	-	Owner
Akshay Y	-	Owner	Owner	Support	-	Owner	-	-
Sumit Yadav	-	-	-	-	Owner	-	Owner	Support
Adil Mirza	-	Support	Support	Owner	-	-	-	-
Harsh Raj	Support	-	-	-	Support	Support	Support	-

Declaration: We confirm that the above contribution details are accurate and verifiable through GitHub commit history and pull request records.

Team Lead: Deepak Mishra

Date: 1st May 2026